组装：Model

NoteKing | 图文讲义

Author / Channel	MiniMind / Mokio
Duration	11m43s
Generated by	NoteKing — Video to Illustrated Notes
Date	2026-04-03
Video Link	https://www.bilibili.com/video/BV1T2k6BaEeC
GitHub	github.com/bcefghj/noteking
Xiaohongshu	bcefghj

目录

1 课程概述 **2**

2 1. Transformer 架构回顾 **2**

2.1 1.1 Transformer 整体架构 3

3 2. 位置编码（Positional Encoding） **4**

3.1 2.1 位置编码实现 4

3.2 2.2 位置编码的直观理解 6

4 3. 多头注意力机制（Multi-Head Attention） **6**

4.1 3.1 缩放点积注意力 7

4.2 3.2 多头注意力完整实现 7

5 4. 前馈神经网络（Feed-Forward Network） **10**

5.1 4.1 FFN 实现 10

5.2 4.2 FFN 的深入理解 11

6 5. 层归一化与残差连接 **12**

6.1 5.1 层归一化实现 12

7 6. 编码器层实现 **13**

8 7. 解码器层实现 **15**

8.1 7.1 解码器掩码机制 16

9 8. 完整 Transformer 模型组装 **17**

9.1 8.1 模型参数统计 18

10 9. 训练与推理流程 **18**

10.1 9.1 训练阶段 18

10.2 9.2 推理阶段（Greedy Decoding） 19

11 10. 代码整合与测试 **20**

12 课程总结 **21**

第 17 集：组装：Model

第 17 课：组装：Model —Transformer 完整实现

课程：MiniMind - PyTorch 从零手敲大模型 **时长**：11 分 43 秒 **Episode**：17/26

---

1 课程概述

本节课将把之前实现的各个模块组合起来，构建一个完整的 Transformer 模型。我们将实现编码器（Encoder）、解码器（Decoder）以及完整的模型架构，为后续的语言模型训练奠定基础。

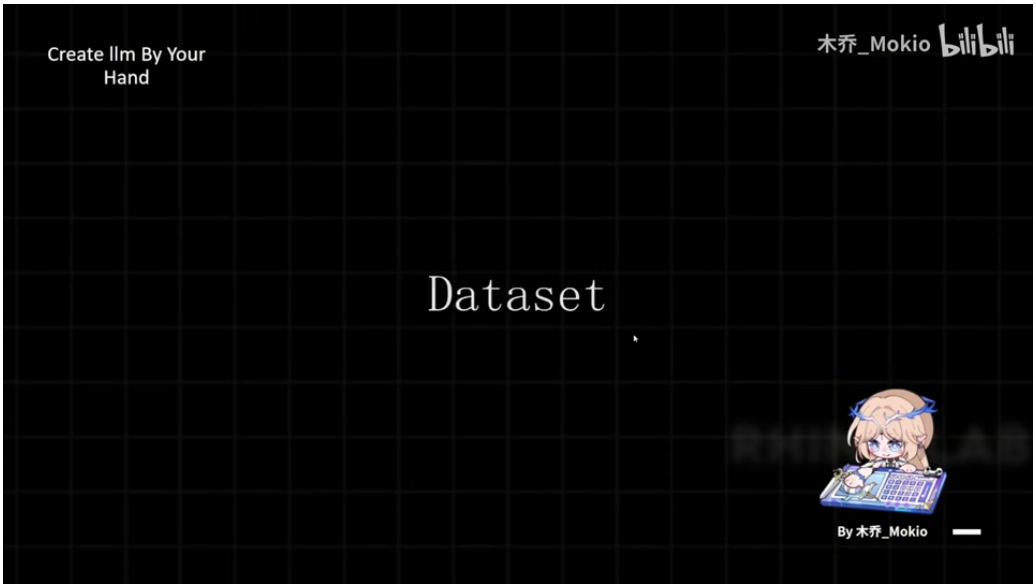

图 1: Video 00:10 [1]

---

2 1. Transformer 架构回顾

核心概念

Transformer 是一种完全基于注意力机制（Attention Mechanism）的序列到序列（Seq2Seq）模型，由 Vaswani 等人于 2017 年在论文《Attention Is All You Need》中首次提出。它摒弃了传统的 RNN 和 CNN 结构，采用自注意力机制实现并行计算，大幅提升了训练效率。

[1] video timestamp 00:10

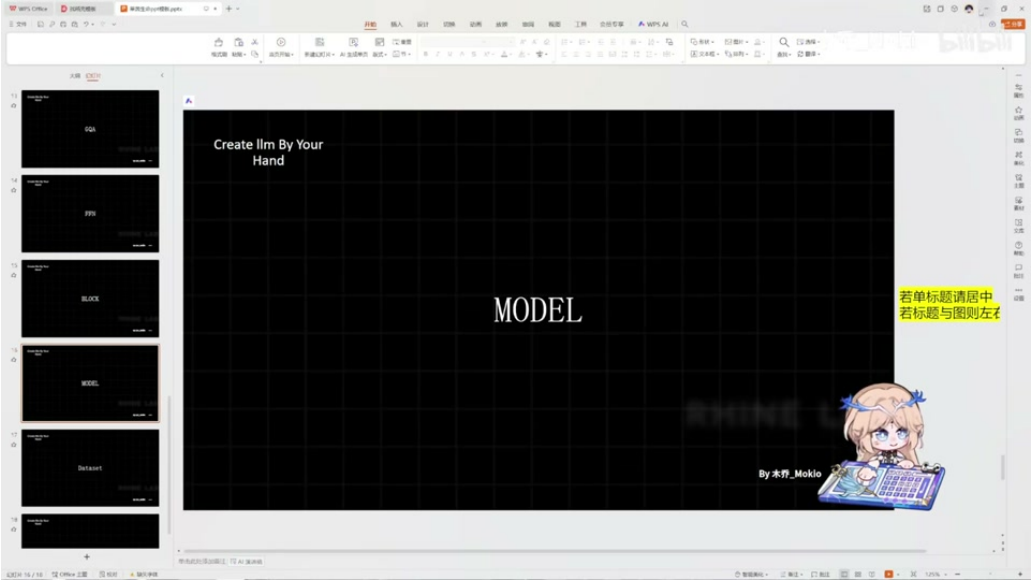

图 2: Video 00:13 [2]

2.1 1.1 Transformer 整体架构

Transformer 由**编码器**和**解码器**两大部分组成：

$$Transformer = Encoder \times N + Decoder \times N$$

- **编码器**：将输入序列转换为连续的表示
- **解码器**：基于编码器输出和已生成的部分序列，生成目标序列

背景知识

典型配置：

- 编码器层数：$N = 6$
- 解码器层数：$N = 6$
- 隐藏层维度：$d_{model} = 512$
- 注意力头数：$h = 8$
- 前馈网络维度：$d_{ff} = 2048$

[2] video timestamp 00:13

3 2. 位置编码（Positional Encoding）

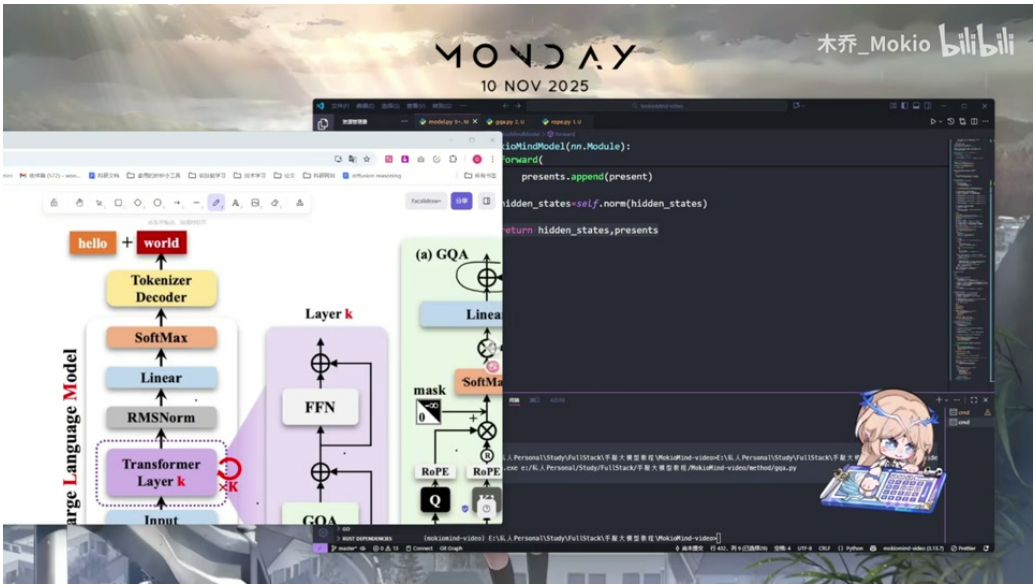

图 3: Video 11:42 [3]

核心概念

由于自注意力机制是位置无关的（Permutation Invariant），需要显式地向模型注入序列中 token 的位置信息。Transformer 使用正弦和余弦函数生成位置编码：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

3.1 2.1 位置编码实现

```python
import torch
import torch.nn as nn
import math

class PositionalEncoding(nn.Module):
    """位置编码层 - 为序列中的每个位置添加位置信息"""

    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        # 创建位置编码矩阵
        pe = torch.zeros(max_len, d_model)
```

[3] video timestamp 11:42

```python
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )

        # 偶数位置使用sin, 奇数位置使用cos
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)  # [1, max_len, d_model]

        # 注册为缓冲区（非参数）
        self.register_buffer('pe', pe)

    def forward(self, x):
        """
        Args:
            x: [batch_size, seq_len, d_model]
        """
        x = x + self.pe[:, :x.size(1), :]
        return self.dropout(x)
```

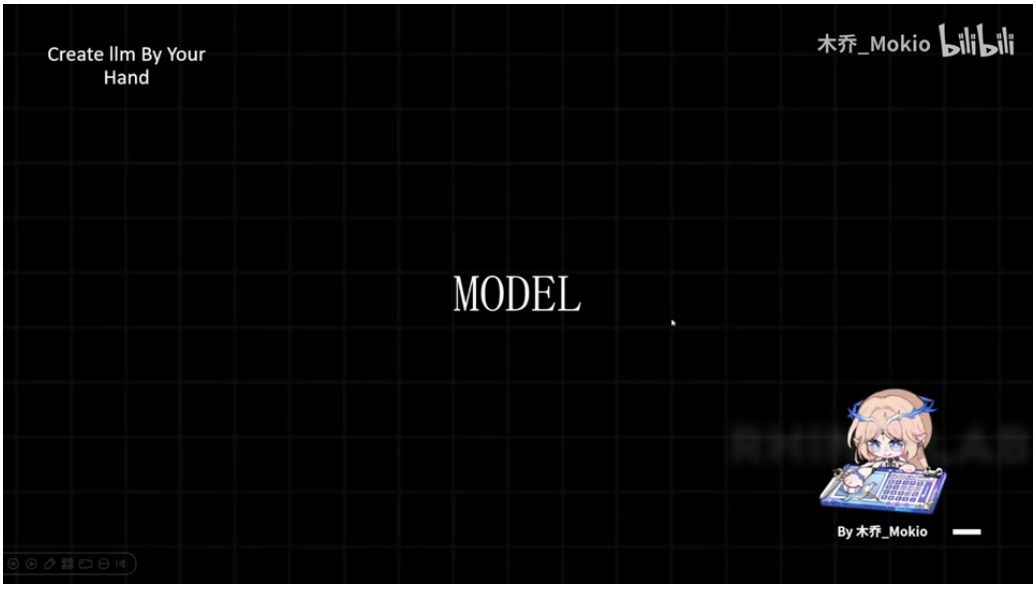

图 4: Video 00:03 [4]

[4] video timestamp 00:03

3.2 2.2 位置编码的直观理解

易错点

1. **维度匹配**：确保位置编码的维度与输入嵌入维度完全一致
2. **可学习 vs 固定**：位置编码可以是固定的（Sinusoidal）也可以是可学习的，Transformer 原始论文使用固定编码
3. **外推能力**：固定位置编码对外推长度的处理有局限

$$PE_{(pos+k,2i)} = \sin(pos+k) = \sin(pos)\cos(k) + \cos(pos)\sin(k)$$

这种编码方式允许模型学习相对位置关系，因为存在线性变换关系。

---

4 3. 多头注意力机制（Multi-Head Attention）

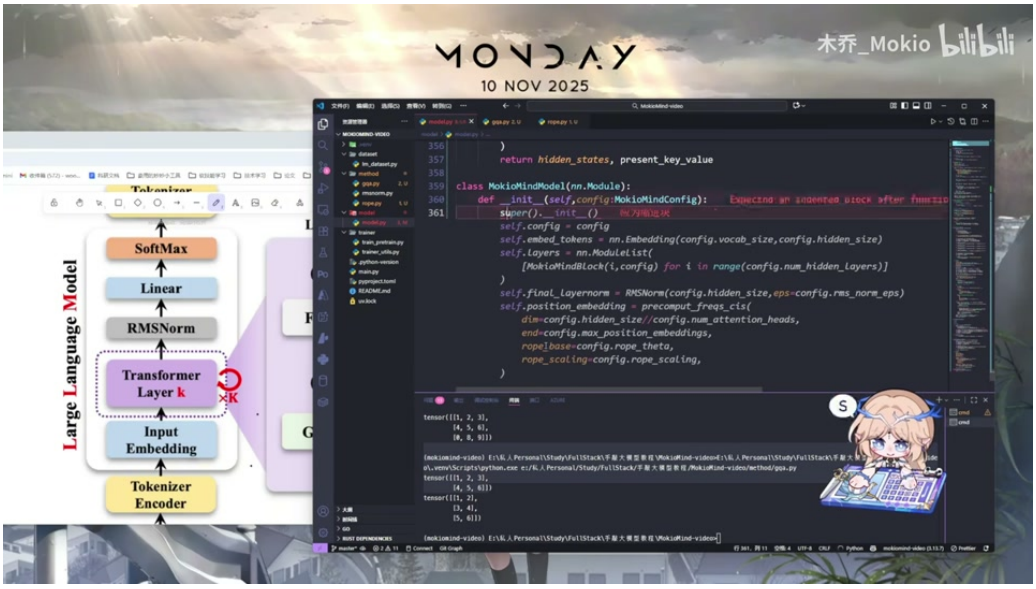

图 5: Video 00:42 [5]

核心概念

多头注意力将输入分别投影到 h 个不同的子空间，每个头独立计算注意力，然后拼接结果：

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

其中 $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

[5] video timestamp 00:42

4.1 3.1 缩放点积注意力

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

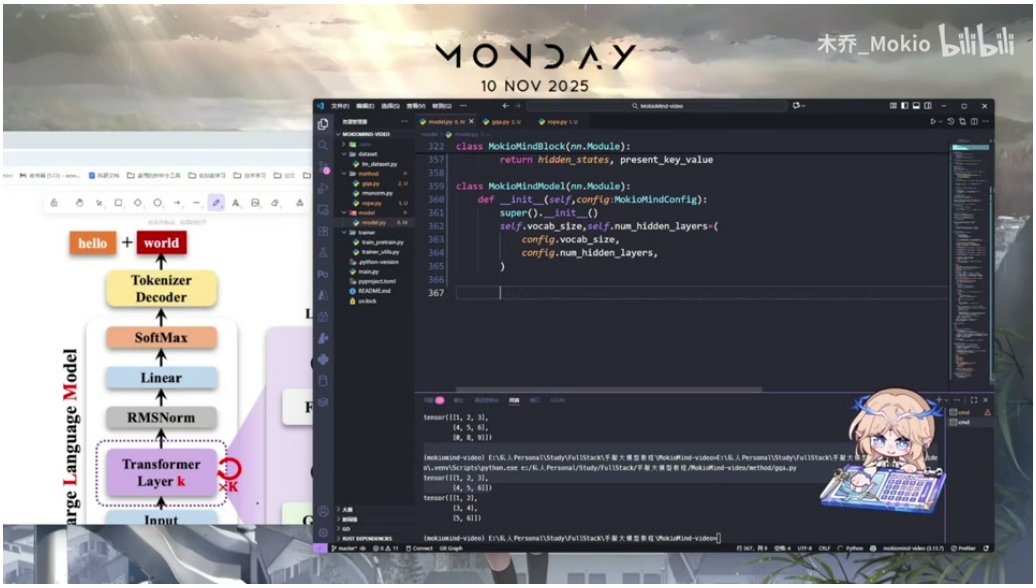

图 6: Video 01:21 [6]

缩放因子 $\sqrt{d_k}$ 用于防止点积值过大导致 softmax 梯度消失。

4.2 3.2 多头注意力完整实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    """多头注意力机制"""

    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0, "d_model必须能被num_heads整除"

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        # 线性投影层
        self.W_q = nn.Linear(d_model, d_model)
```

[6] video timestamp 01:21

```python
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

        self.dropout = nn.Dropout(dropout)

    def forward(self, query, key, value, mask=None):
        """
        Args:
            query: [batch_size, seq_len, d_model]
            key: [batch_size, seq_len, d_model]
            value: [batch_size, seq_len, d_model]
            mask: 注意力掩码
        Returns:
            output: [batch_size, seq_len, d_model]
            attention_weights: [batch_size, num_heads, seq_len, seq_len]
        """
        batch_size = query.size(0)
        seq_len = query.size(1)

        # 线性投影并分头
        Q = self.W_q(query).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(key).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(value).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

        # 缩放点积注意力
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)

        # 应用掩码
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attention_weights = F.softmax(scores, dim=-1)
        attention_weights = self.dropout(attention_weights)

        # 计算输出
        context = torch.matmul(attention_weights, V)

        # 合并多头结果
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.d_model)
        output = self.W_o(context)

```

```
        return output, attention_weights
```

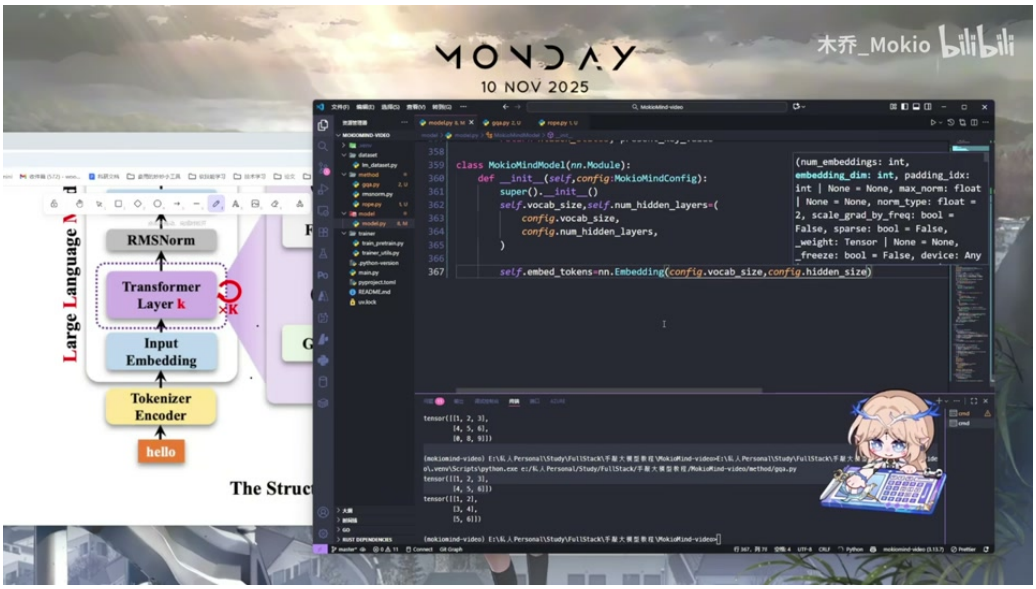

图 7: Video 02:00 [7]

背景知识

三种注意力类型:

1. **自注意力（Self-Attention）**：$Q = K = V = X$，输入序列与自身的注意力
2. **源注意力（Source Attention）**：编码器到解码器的注意力
3. **掩码注意力（Masked Attention）**：解码器中防止看到未来信息

[7] video timestamp 02:00

5 4. 前馈神经网络（Feed-Forward Network）

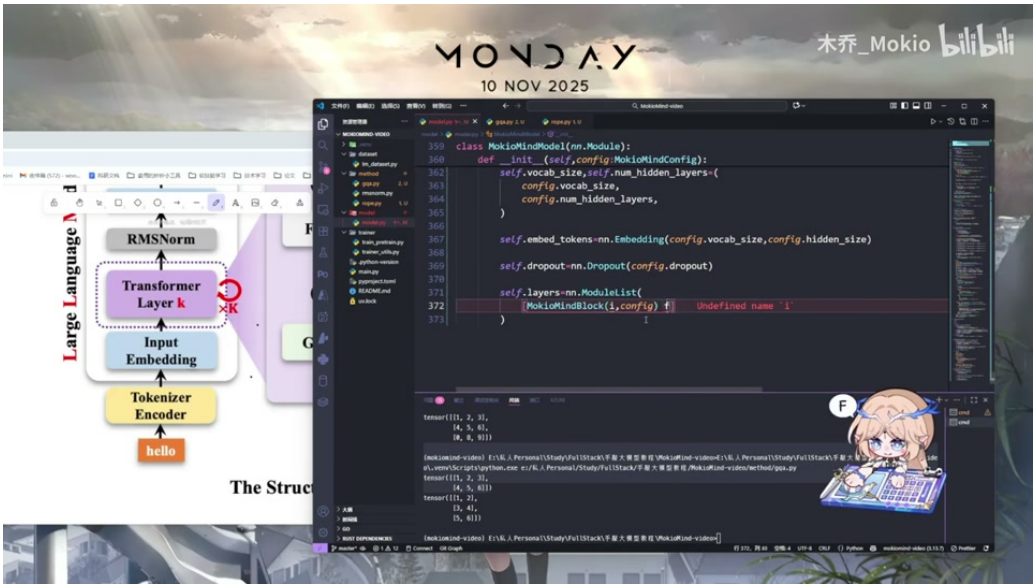

图 8: Video 02:39 [8]

核心概念

每个 Transformer 层包含一个 Position-wise 前馈网络，由两个线性变换组成，中间使用 ReLU 激活：

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

5.1 4.1 FFN 实现

```python
class PositionwiseFeedForward(nn.Module):
    """位置-wise 前馈神经网络"""

    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.w_1 = nn.Linear(d_model, d_ff)
        self.w_2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)
        self.activation = nn.ReLU()

    def forward(self, x):
        return self.w_2(self.dropout(self.activation(self.w_1(x))))
```

[8] video timestamp 02:39

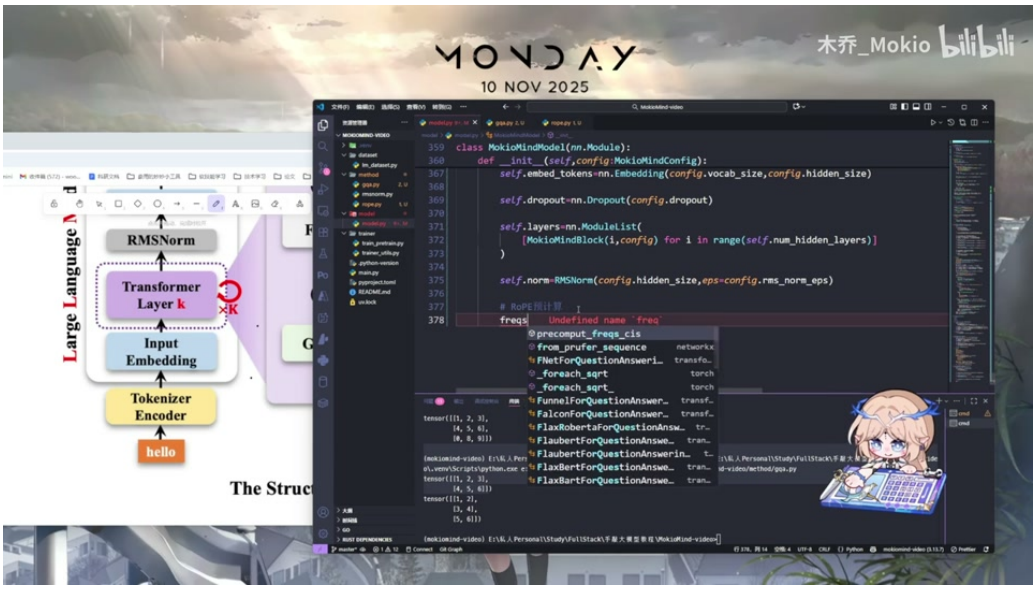

图 9: Video 03:18 [9]

5.2 4.2 FFN 的深入理解

组件	维度	说明
输入	$d_\{model\} = 512$	与注意力层输出维度一致
隐藏层	$d_\{ff\} = 2048$	4 倍扩展，提供模型容量
输出	$d_\{model\} = 512$	恢复原始维度
激活函数	ReLU/GELU	引入非线性

[9] video timestamp 03:18

6 5. 层归一化与残差连接

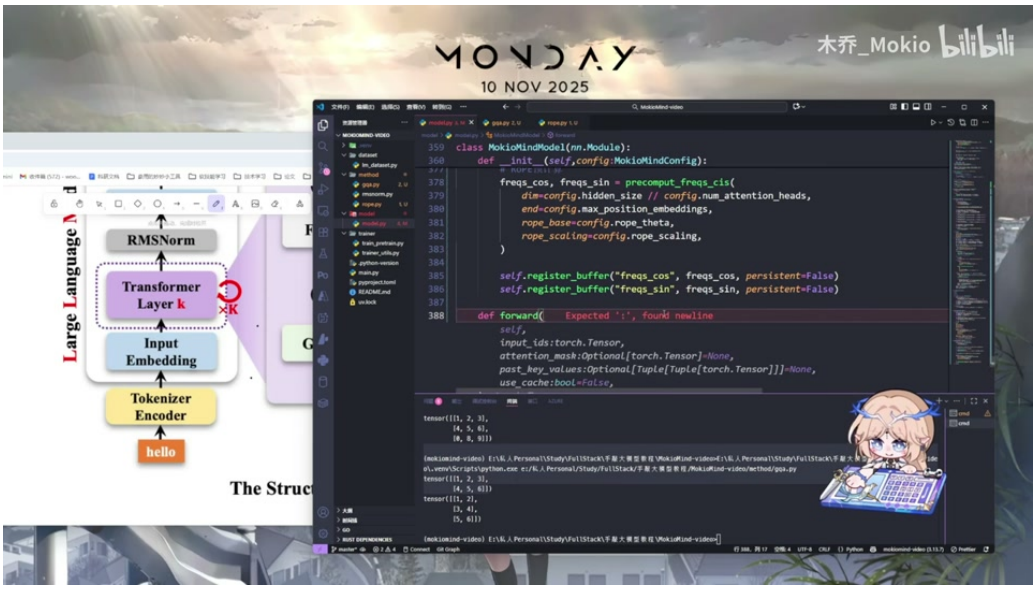

图 10: Video 03:57 [10]

核心概念

Transformer 使用 **Post-LN（Layer Normalization）** 结构：

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

6.1 5.1 层归一化实现

```python
class LayerNorm(nn.Module):
    """层归一化"""

    def __init__(self, features, eps=1e-6):
        super().__init__()
        self.gamma = nn.Parameter(torch.ones(features))
        self.beta = nn.Parameter(torch.zeros(features))
        self.eps = eps

    def forward(self, x):
        mean = x.mean(-1, keepdim=True)
        std = x.std(-1, keepdim=True)
        return self.gamma * (x - mean) / (std + self.eps) + self.beta
```

易错点

[10] video timestamp 03:57

Pre-LN vs Post-LN：

- 原始 Transformer 使用 Post-LN（归一化在残差连接之后）
- 现代实现常使用 Pre-LN（归一化在残差连接之前）
- Pre-LN 在训练稳定性上通常表现更好

---

7 6. 编码器层实现

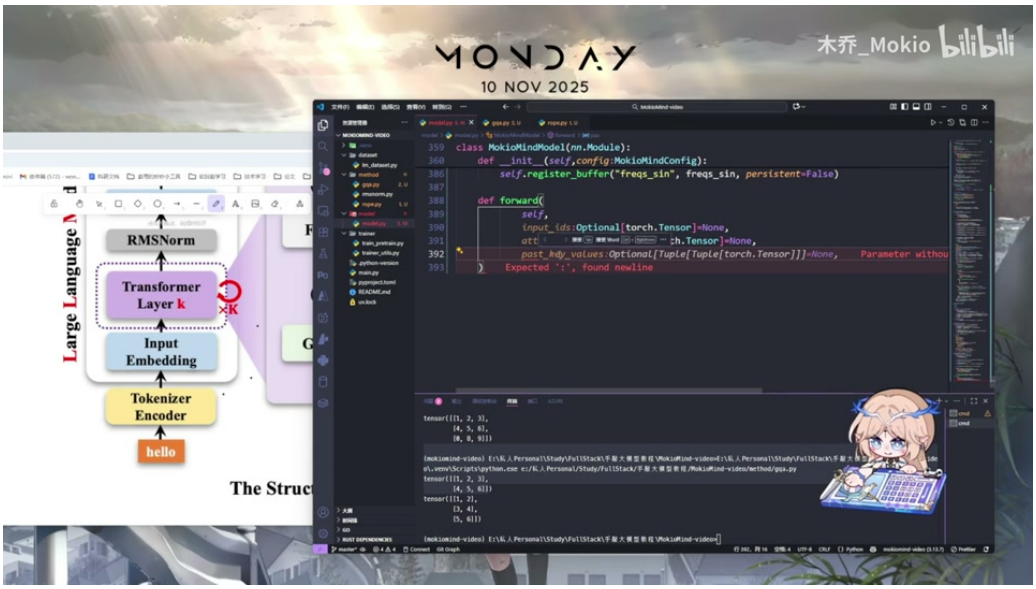

图 11: Video 04:36 [11]

```python
class EncoderLayer(nn.Module):
    """Transformer编码器层"""

    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attention = MultiHeadAttention(d_model, num_heads, dropout
            )
        self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)

    def forward(self, x, mask=None):
```

[11] video timestamp 04:36

```python
        # 自注意力 + 残差连接
        attn_output, _ = self.self_attention(x, x, x, mask)
        x = self.norm1(x + self.dropout1(attn_output))

        # 前馈网络 + 残差连接
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout2(ff_output))

        return x


class Encoder(nn.Module):
    """Transformer编码器"""

    def __init__(self, num_layers, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([
            EncoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, mask=None):
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

8 7. 解码器层实现

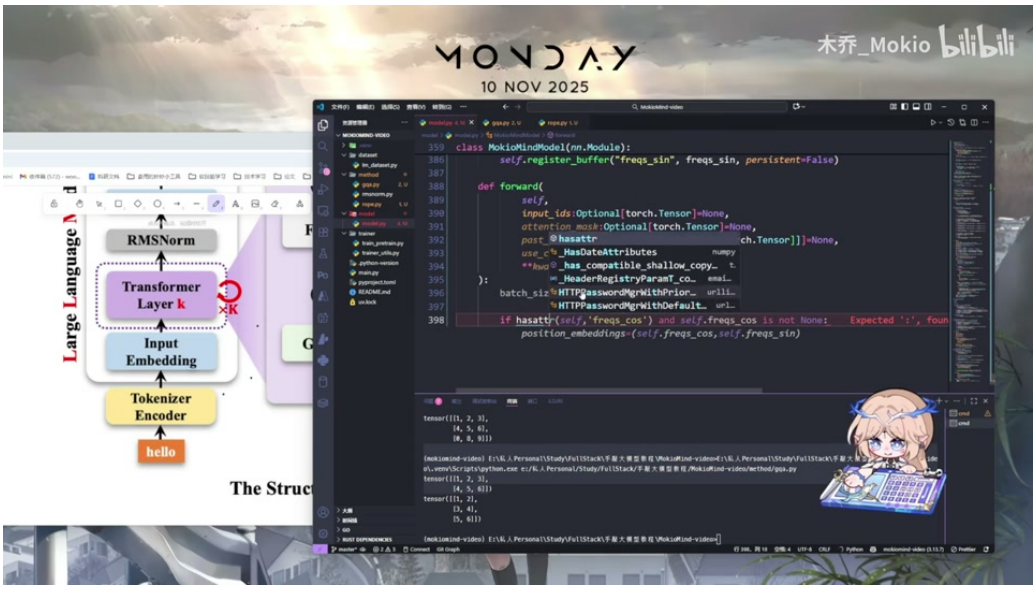

图 12: Video 05:15 [12]

```python
class DecoderLayer(nn.Module):
    """Transformer解码器层"""

    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attention = MultiHeadAttention(d_model, num_heads, dropout)
        self.cross_attention = MultiHeadAttention(d_model, num_heads, dropout)
        self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)

        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)

        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)
        self.dropout3 = nn.Dropout(dropout)

    def forward(self, x, encoder_output, src_mask=None, tgt_mask=None):
        # 自注意力（掩码）
        attn1, _ = self.self_attention(x, x, x, tgt_mask)
        x = self.norm1(x + self.dropout1(attn1))

        # 交叉注意力（编码器-解码器）
```

[12] video timestamp 05:15

```
        attn2, _ = self.cross_attention(x, encoder_output, encoder_output,
            src_mask)
        x = self.norm2(x + self.dropout2(attn2))

        # 前馈网络
        ff_output = self.feed_forward(x)
        x = self.norm3(x + self.dropout3(ff_output))

        return x


class Decoder(nn.Module):
    """Transformer解码器"""

    def __init__(self, num_layers, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([
            DecoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, encoder_output, src_mask=None, tgt_mask=None):
        for layer in self.layers:
            x = layer(x, encoder_output, src_mask, tgt_mask)
        return self.norm(x)
```

8.1 7.1 解码器掩码机制

```
def create_padding_mask(seq, pad_idx=0):
    """创建填充掩码"""
    return (seq != pad_idx).unsqueeze(1).unsqueeze(2)

def create_look_ahead_mask(size):
    """创建前瞻掩码（防止看到未来信息）"""
    mask = torch.triu(torch.ones(size, size), diagonal=1).type(torch.uint8)
    return mask == 0

def create_combined_mask(tgt):
    """组合掩码：填充掩码 + 前瞻掩码"""
    padding_mask = create_padding_mask(tgt)
    look_ahead_mask = create_look_ahead_mask(tgt.size(1))
    combined_mask = look_ahead_mask & padding_mask
    return combined_mask
```

9 8. 完整 Transformer 模型组装

```python
class Transformer(nn.Module):
    """完整的Transformer模型"""

    def __init__(
        self,
        src_vocab_size,
        tgt_vocab_size,
        d_model=512,
        num_heads=8,
        num_encoder_layers=6,
        num_decoder_layers=6,
        d_ff=2048,
        dropout=0.1,
        max_len=5000
    ):
        super().__init__()

        # 嵌入层
        self.src_embedding = nn.Embedding(src_vocab_size, d_model)
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)

        # 位置编码
        self.pos_encoder = PositionalEncoding(d_model, max_len, dropout)
        self.pos_decoder = PositionalEncoding(d_model, max_len, dropout)

        # 编码器和解码器
        self.encoder = Encoder(num_encoder_layers, d_model, num_heads, d_ff,
            dropout)
        self.decoder = Decoder(num_decoder_layers, d_model, num_heads, d_ff,
            dropout)

        # 输出投影层
        self.output_proj = nn.Linear(d_model, tgt_vocab_size)

        # 缩放因子
        self.d_model = d_model
        self._init_weights()

    def _init_weights(self):
        """权重初始化"""
        for p in self.parameters():
```

```
            if p.dim() > 1:
                nn.init.xavier_uniform_(p)

    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        # 源序列编码
        src_emb = self.pos_encoder(self.src_embedding(src) * math.sqrt(self.d_model))
        encoder_output = self.encoder(src_emb, src_mask)

        # 目标序列解码
        tgt_emb = self.pos_decoder(self.tgt_embedding(tgt) * math.sqrt(self.d_model))
        decoder_output = self.decoder(tgt_emb, encoder_output, src_mask, tgt_mask)

        # 投影到词汇表
        output = self.output_proj(decoder_output)

        return output
```

9.1 8.1 模型参数统计

组件	参数量
嵌入层	$VBACKSLASH_PLACEHOLDERtimesd_\{model\}BACKSLASH_PLACEHOLDERt$
注意力层	$4BACKSLASH_PLACEHOLDERtimesd_\{model\}\hat{}2BACKSLASH_PLACEHOLDER$
前馈网络	$2BACKSLASH_PLACEHOLDERtimesd_\{model\}BACKSLASH_PLACEHOLDERti$
归一化层	$2BACKSLASH_PLACEHOLDERtimesd_\{model\}$

---

10 9. 训练与推理流程

10.1 9.1 训练阶段

```
def train_step(model, optimizer, criterion, src, tgt):
    model.train()

    # 创建掩码
    src_mask = create_padding_mask(src)
    tgt_mask = create_combined_mask(tgt)

    # 前向传播
    output = model(src, tgt[:, :-1], src_mask, tgt_mask)

```

```
    # 计算损失
    loss = criterion(
        output.reshape(-1, output.size(-1)),
        tgt[:, 1:].reshape(-1)
    )

    # 反向传播
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    return loss.item()
```

10.2 9.2 推理阶段（Greedy Decoding）

```
def greedy_decode(model, src, max_len, start_token, end_token, src_mask=None):
    model.eval()

    # 编码源序列
    src_emb = model.pos_encoder(model.src_embedding(src) * math.sqrt(model.d_model))
    encoder_output = model.encoder(src_emb, src_mask)

    # 解码
    decoder_input = torch.tensor([[start_token]], device=src.device)

    for _ in range(max_len):
        tgt_emb = model.pos_decoder(model.tgt_embedding(decoder_input) * math.sqrt(model.d_model))
        decoder_output = model.decoder(tgt_emb, encoder_output, src_mask, None)
        output = model.output_proj(decoder_output)

        # 选择概率最高的token
        next_token = output[:, -1, :].argmax(dim=-1, keepdim=True)
        decoder_input = torch.cat([decoder_input, next_token], dim=1)

        if next_token.item() == end_token:
            break

    return decoder_input
```

11 10. 代码整合与测试

```python
# 模型配置
config = {
    'src_vocab_size': 10000,
    'tgt_vocab_size': 10000,
    'd_model': 512,
    'num_heads': 8,
    'num_encoder_layers': 6,
    'num_decoder_layers': 6,
    'd_ff': 2048,
    'dropout': 0.1,
    'max_len': 200
}

# 创建模型
model = Transformer(**config)

# 统计参数量
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"总参数量: {total_params:,}")
print(f"可训练参数: {trainable_params:,}")

# 测试前向传播
batch_size = 4
src_seq_len = 50
tgt_seq_len = 30

src = torch.randint(0, config['src_vocab_size'], (batch_size, src_seq_len))
tgt = torch.randint(0, config['tgt_vocab_size'], (batch_size, tgt_seq_len))

output = model(src, tgt)
print(f"输出形状: {output.shape}")  # [batch_size, tgt_seq_len, tgt_vocab_size]
```

12 课程总结

核心要点

1. **Transformer 由编码器和解码器组成**，每部分包含多层堆叠的自注意力层和前馈网络
2. **位置编码**通过正弦余弦函数注入序列位置信息，使模型能够区分不同位置的 token
3. **多头注意力**将注意力分散到多个子